

Unit 1 - Chapter 1 – Lesson 1 – Control with joystick

1. Objectives:

- Know how joysticks control movement electronically.
- Identify the parts of a basic joystick circuit.
- Build a circuit to read joystick signals with Arduino.
- Understand how joystick movement creates changing values.
- Explain how joystick signals can control a robot arm.
- Start designing a way to control a robot arm with a joystick.

2. Challenge Question of the Lesson:

- How can we use a joystick to tell a robot arm how to move?

Short Answer: By reading the changing electrical signals from the joystick and using those signals to control the motors in the robot arm.

3. SEL Relations:

- Self-Awareness: Think about how you control things with your hands and how the joystick is similar or different.
- Self-Management: Stay focused while building the circuit and writing the code; be patient when testing.
- Social Awareness: Imagine how precise control can help robots do useful tasks for people.
- Relationship Skills: Share ideas about how to control the robot arm with classmates.
- Responsible Decision-Making: Consider how accurate control is important for robots doing tasks safely.

1. Engage:

Teacher Role:

- Begin by asking students about their experiences with video games or other devices that use joysticks (e.g., some remote-controlled cars, arcade games).
- Show examples of different types of joysticks (simple analog joysticks, more complex gaming controllers).
- Pose the "Let's Think" questions to the class and encourage them to share their initial ideas about how joysticks sense movement.
- Briefly discuss how the movement of the joystick translates into actions on the screen or in the real world (e.g., moving a character, controlling a robot's joint).
- Hint that understanding how joysticks work is a key step in learning how to control robotic arms.

Expected From Students:

- Share their experiences with joysticks and related devices.
- Offer initial guesses and ideas about the internal mechanisms of a joystick.
- Show curiosity about how joystick movement is converted into control signals.
- Recognize the connection between joystick control and the potential control of robotic arms.

Preview with a Hook:

- **Strategy:** Use a short demonstration or a relatable scenario.
- **Implementation:**
 - **Demonstration:** If possible, show a simple video of someone using a joystick to control a robot (even a virtual one).
 - **Scenario:** Describe a situation where a robotic arm needs to pick up an object and how a joystick could be used to precisely control its movement (up, down, left, right, grab, release).
 - Ask: "How do you think the person's hand movements on the joystick are telling the robot arm what to do?"

Strategies to use with this part:

- **Class Discussion:** Facilitate an open discussion about where students have seen or used joysticks.
 - **Quick Draw:** Have students quickly sketch what they think might be inside a joystick.
 - **Analogy:** Compare the joystick to other input devices they are familiar with (like a mouse or arrow keys) and discuss how they translate physical movement into digital signals.
-

2. Explore :

Teacher Role:

- Clearly outline the materials needed for this hands-on activity, ensuring students understand what each component is and its role.
- Guide students through each step of the process, emphasizing safety precautions when working with electrical components and mechanical parts.

- Provide visual aids or demonstrations for each step, such as how to mount the servos, wire them to the Arduino, and connect the power supply.
- Explain the basic structure of the provided Arduino code, focusing on how the Servo library is used to control the angles of the servo motors.
- Assist students in uploading the code to their Arduino boards and observing the resulting movements of the robotic arm's joints.
- Facilitate the testing and calibration phase, encouraging students to identify any issues and make necessary adjustments to the code or the physical setup.
- Explain that this basic control of the arm's joints is the foundation for controlling it with the joystick in the next stage.

Expected From Students:

- Gather and organize the required materials for the robotic arm assembly and control.
- Follow the step-by-step instructions to mount the servo motors to the robotic arm.
- Connect the servo motor wires correctly to the Arduino microcontroller, paying attention to power, ground, and signal connections.
- Connect the power supply appropriately to the servos and the Arduino.
- Open the Arduino IDE on their computers.
- Understand the basic structure of the provided Arduino code, particularly the use of the Servo library and the `attach()` and `write()` functions.
- Upload the code successfully to the Arduino board.

- Observe the movement of the robotic arm's joints as commanded by the code.
- Identify any issues with the movement (e.g., incorrect direction, limited range) and attempt to troubleshoot them through code adjustments or physical adjustments to the servo mounting.

Detailed Breakdown of the "Do It!" Steps for Teacher Guidance:

- **Mount Servo Motors:**

- **Guidance:** Show students how the servos align with the joints of their specific robotic arm kit. Emphasize secure attachment using the provided screws or mounting hardware to prevent slippage during movement.
- **Safety:** Remind students to handle the robotic arm and servo motors carefully to avoid damage.

- **Wire Servos:**

- **Guidance:** Clearly illustrate the color-coding of servo wires (usually power - red/orange, ground - brown/black, signal - yellow/white). Explain which Arduino pins correspond to power (5V), ground (GND), and digital signal (pins 9, 10, 11 in the example). Stress the importance of correct polarity.
- **Power Supply:** Explain that servos often require a separate power supply, especially if multiple servos are used, to avoid overloading the Arduino's power. Guide them on connecting the external power supply to both the servos (power and ground) and the Arduino (ground).
- **Connection Diagram:** Provide a clear wiring diagram specific to their robotic arm and the Arduino.

- **Write Code:**

- **Guidance:** Open the Arduino IDE and explain the purpose of the `#include <Servo.h>` line (to access the Servo library functions).
- **attach() Function:** Explain that `shoulder.attach(9);` tells the Arduino that the signal wire of the servo named `shoulder` is connected to digital pin 9. Guide them to adjust the pin numbers if their wiring is different.
- **loop() Function:** Break down the for loops. Explain that `for (int pos = 0; pos <= 180; pos++)` creates a variable `pos` that counts from 0 to 180. The `shoulder.write(pos);` command tells the servo to move to the angle specified by `pos`. The `delay(15);` command makes the program wait for a short time, controlling the speed of movement. Explain the second for loop does the reverse movement.
- **Customization:** Encourage students to experiment with changing the `delay()` value to alter the speed of the arm's movement and the range of motion (by changing the limits in the for loop, cautiously).

● Upload Code:

- **Guidance:** Ensure students know how to connect their Arduino to the computer via a USB cable and how to select the correct board and port in the Arduino IDE (Tools > Board and Tools > Port).
- **Upload Button:** Show them the "Upload" button (the right-pointing arrow) and explain that this sends the code to the Arduino's memory.

● Test and Calibrate:

- **Guidance:** Instruct students to power on the Arduino and observe the movement of the robotic arm.

- **Troubleshooting:** Guide them to identify common issues:
 - **No Movement:** Check power connections, wiring to the Arduino, and if the code was uploaded successfully.
 - **Erratic Movement:** Check for loose wires or incorrect servo connections.
 - **Limited Range:** Ensure the servo's physical range of motion isn't being restricted by the arm's mechanics and that the code's `map()` function (if used later) is correctly scaling the input.
 - **Incorrect Direction:** Students might need to reverse the `map()` function's output range in later steps if the joystick controls the arm in the opposite way they intend.
-

Materials:

- Robotic arm
- Servo motors
- Microcontroller (e.g., Arduino)
- Power supply
- Wiring and connectors
- Mounting hardware
- Programming software (e.g., Arduino IDE)

Steps:

1. Mount Servo Motors:

- Attach servos to the robotic arm joints using mounting hardware.

2. Wire Servos:

- Connect servo wires (power, ground, signal) to the microcontroller.
- Connect power supply to servos and controller.

3. Write Code:

- Open Arduino IDE.
- Use the Servo library to write code (see example below).
- Example Code:

cpp

Copy code

```
#include <Servo.h>
```

```
Servo shoulder, elbow, wrist;
```

```
void setup() {
```

```
  shoulder.attach(9);
```

```
  elbow.attach(10);
```

```
  wrist.attach(11);
```

```
}
```

```
void loop() {
```

```
  for (int pos = 0; pos <= 180; pos++) {
```

```
    shoulder.write(pos);
```

```
    elbow.write(pos);
```

```
    wrist.write(pos);
```



```
    delay(15);  
}  
for (int pos = 180; pos >= 0; pos--) {  
    shoulder.write(pos);  
    elbow.write(pos);  
    wrist.write(pos);  
    delay(15);  
}  
}
```

4. Upload Code:

- Connect the microcontroller to your computer and upload the code.

5. Test and Calibrate:

- Power on and observe servo movements.
 - Adjust code or setup as needed.
-

3. Evaluate (or Assess):

Teacher Role:

- Explain to students that this is a chance to check what they've learned about joysticks and how they can be used to control things, like the robotic arm they're working on.
- Go over the instructions for choosing the correct answer for each question.
- Remind them to think about the discussions from the beginning of the lesson and what they learned from the video and reading materials in the "Explore" section.
- Observe students as they work, and be ready to help if they don't understand a question, but don't give away the answers.

- After everyone has finished, review the answers together and talk about why the correct answer is right. This helps clear up any confusion.

Expected From Students:

- Read each question carefully and think about what it's asking.
- Use what they've learned about joysticks to choose the best answer for each question.
- Show that they understand what joysticks are used for and how they work.

Focus - Assessment Questions and Teacher Guidance:

- **Question 1: What is the primary function of a joystick module?**
 - A) To display images.
 - B) To control motion.
 - C) To play music.
 - D) To charge batteries.
 - **Correct Answer: B) To control motion.**
 - **Teacher Guidance:** Remind students about the examples of joysticks they discussed at the start (video games, remote control cars). Emphasize that the main purpose is to tell something else how to move.
- **Question 2: In which of the following applications are joysticks NOT commonly used?**
 - A) Video game controllers.
 - B) Remote-controlled cars.
 - C) Microwave ovens.
 - D) Flight simulators.
 - **Correct Answer: C) Microwave ovens.**
 - **Teacher Guidance:** Ask students to think about how they control a microwave. It uses buttons or a keypad, not a stick for movement. The other options all commonly use joysticks for controlling movement or actions.
- **Question 3: What type of signal does a joystick module produce?**
 - A) Audio signal.
 - B) Visual signal.

- o C) Electronic signal.
 - o D) Magnetic signal.
 - o **Correct Answer: C) Electronic signal.**
 - o **Teacher Guidance:** Connect this back to the "Explore" activity where they might have used a potentiometer. Explain that the joystick changes electrical values (voltage, resistance), which are electronic signals that the Arduino can read.
 - **Question 4: How does a joystick typically detect movement?**
 - o A) Through a camera.
 - o B) Using potentiometers or sensors.
 - o C) With a microphone.
 - o D) By measuring temperature changes.
 - o **Correct Answer: B) Using potentiometers or sensors.**
 - o **Teacher Guidance:** Refer to the "Explore" video or reading about what's inside a joystick. Explain that potentiometers (variable resistors) or other types of sensors inside the joystick change their electrical properties when the stick is moved, and this change is how the movement is detected.
-

4. Practice:

Teacher Role:

- Clearly outline the specific materials needed for this activity, including the KY-023 joystick module, Arduino Uno, breadboard, jumper wires, and power source.
- Guide students through the step-by-step process of connecting the joystick module to the Arduino on the breadboard, emphasizing the correct wiring for VCC, GND, VRx, VRy, and the optional SW (button). Provide a clear breadboard layout diagram if possible.
- Explain the provided Arduino code, focusing on how it reads the analog values from the joystick's X and Y axes and the digital state of the button.
- Instruct students on how to open the Serial Monitor in the Arduino IDE to observe the output.
- Guide students in testing the joystick's functionality by moving it in different directions and pressing the button, observing the corresponding changes in the X, Y values, and button state in the Serial Monitor.
- Explain that these observed values will be used in the next step to control the servo motors of the robotic arm.

Expected From Students:

- Gather the specified materials for building the joystick circuit.
- Follow the wiring diagram to connect the KY-023 joystick module to the Arduino on the breadboard correctly.
- Open the provided Arduino code in the Arduino IDE.
- Understand the purpose of the `analogRead()` function for the joystick axes and `digitalRead()` for the button.
- Upload the code successfully to the Arduino board.
- Open the Serial Monitor and observe the changing X and Y values as they move the joystick.
- Observe the change in the button state (Pressed/Released) when they press the joystick button.
- Understand that these changing numerical values represent the joystick's input and can be used to control the robotic arm's movements.

Detailed Breakdown of the "Do It!" Steps for Teacher Guidance:

- **Connect Joystick to Arduino:**
 - **Guidance:** Provide a clear breadboard layout showing the connections. Emphasize the importance of connecting VCC and GND correctly to avoid damaging the components.
 - **KY-023 Pinout:** Clearly explain the function of each pin on the KY-023 joystick module (VCC, GND, VRx, VRy, SW).
 - **Wiring Instructions:**
 - Connect Joystick VCC to Arduino 5V using a jumper wire.
 - Connect Joystick GND to Arduino GND using a jumper wire.
 - Connect Joystick VRx to Arduino Analog Pin A0 using a jumper wire.
 - Connect Joystick VRy to Arduino Analog Pin A1 using a jumper wire.
 - (Optional) Connect Joystick SW to Arduino Digital Pin 2 using a jumper wire. Explain that the `INPUT_PULLUP` in the code makes the button read HIGH (Released) when not pressed and LOW (Pressed).
- **Write and Upload Code:**
 - **Guidance:** Open the Arduino IDE and have students copy the provided code.
 - **Code Explanation (for students):**

- `const int joyX = A0;`: Creates a constant named joyX and assigns it the value A0 (the analog pin for the X-axis).
 - `const int joyY = A1;`: Does the same for the Y-axis.
 - `const int buttonPin = 2;`: Assigns digital pin 2 to the button.
 - `Serial.begin(9600);`: Starts communication with the Serial Monitor.
 - `pinMode(buttonPin, INPUT_PULLUP);`: Configures the button pin as an input with a built-in pull-up resistor (simplifies button wiring).
 - **loop() Function:**
 - `int xValue = analogRead(joyX);`: Reads the analog voltage from the joystick's X-axis (0-1023).
 - `int yValue = analogRead(joyY);`: Reads the analog voltage from the joystick's Y-axis (0-1023).
 - `bool buttonState = digitalRead(buttonPin);`: Reads the digital state of the button (HIGH or LOW).
 - `Serial.print()` and `Serial.println()`: Display the X and Y values and the button state on the Serial Monitor.
 - `delay(100);`: Pauses for a short time.
 - **Upload:** Guide students to connect the Arduino to the computer and upload the code.
 - **Observe Joystick Function:**
 - **Guidance:** Instruct students to open the Serial Monitor (Tools > Serial Monitor).
 - **Testing:** Have them move the joystick:
 - Observe how the xValue changes as they move the joystick left and right.
 - Observe how the yValue changes as they move the joystick up and down.
 - Observe how the Button value changes between "Released" and "Pressed" when they push down on the joystick.
 - **Understanding the Output:** Explain that the range of xValue and yValue (around 0 to 1023) represents the joystick's position. These values can now be mapped to control the angles of the servo motors on the robotic arm.
-

Building a Joystick Circuit and Code

Materials:

- Joystick module (e.g., KY-023)
- Arduino Uno
- Breadboard
- Jumper wires
- Power source (e.g., USB cable for Arduino)
- Arduino IDE

Steps:

1. **Connect Joystick to Arduino:**
 - o **VCC:** Joystick VCC to Arduino 5V.
 - o **GND:** Joystick GND to Arduino GND.
 - o **VRx:** Joystick VRx to Arduino A0.
 - o **VRy:** Joystick VRy to Arduino A1.
 - o **SW** (optional): Joystick SW to Arduino D2.
2. **Write and Upload Code:**

cpp

Copy code

```
const int joyX = A0;
```

```
const int joyY = A1;
```

```
const int buttonPin = 2;
```

```
void setup() {
```

```
    Serial.begin(9600);
```

```
    pinMode(buttonPin, INPUT_PULLUP);
```

```
}
```

```
void loop() {
```

```
    int xValue = analogRead(joyX);
```

```
int yValue = analogRead(joyY);

bool buttonState = digitalRead(buttonPin);


Serial.print("X: ");
Serial.print(xValue);
Serial.print(" | Y: ");
Serial.print(yValue);
Serial.print(" | Button: ");
Serial.println(buttonState ? "Released" : "Pressed");


delay(100);

}
```

3. **Observe Joystick Function:**

- o Open Serial Monitor in Arduino IDE.
- o Move joystick and check X, Y values and button state.

Unit 3

Chapter 1 : Programming

Lesson 1: Your first GUI

1. Objectives:

- Understand that computer programs can have visual controls.
- Know what Python and Tkinter are used for in making apps.
- Create a basic window with Python and Tkinter.
- Add sliders to a GUI to control values.
- Understand how slider movements can send commands.
- Start planning how sliders can control a robot arm's actions (gripper, speed, left/right).

2. Challenge Question of the Lesson:

- How can we make a computer screen with sliders to control different parts of our robot arm?

Short Answer: By using programming tools like Python and Tkinter to create visual sliders that send different number values when moved.

3. SEL Relations:

- **Self-Awareness:** Think about how you use sliders on your phone or other devices and how they feel intuitive to control.
- **Self-Management:** Stay focused while writing the Python code and experimenting with the sliders.
- **Social Awareness:** Imagine how a clear and easy-to-use control panel can help people operate complex robots.
- **Relationship Skills:** Share your ideas for the GUI layout and help classmates with their code.
- **Responsible Decision-Making:** Consider how the design of the control panel affects how accurately and safely the robot arm can be used.

1. Engage:

Teacher Role:

- Begin by asking students about their experiences with computer programs that have buttons, sliders, and other interactive elements (GUIs) – think of simple apps or software they use.
- Pose the "Let's Think" question to the class and facilitate a discussion about the advantages of using a computer program with a GUI to control a robotic arm in a factory setting. Guide them to consider aspects like precision, ease of use, storing sequences of movements, and potential for automation.
- Introduce the idea that instead of directly writing code for the Arduino each time, they will learn to create a program on the computer that sends commands to the Arduino to control the robotic arm.
- Mention that they will be learning about a programming language called Python and a tool in Python called Tkinter that helps create these visual interfaces.
- Briefly explain that this is similar to how software interfaces are used to control complex machinery in real-world factories.

Expected From Students:

- Share their experiences with computer programs and GUIs.
- Offer ideas about the benefits of using a computer program to control a robotic arm in a factory.
- Show curiosity about how a computer program can communicate with and control a physical device like their robotic arm.
- Express initial interest in learning about Python and Tkinter.

Preview with a Hook:

- **Strategy:** Show a short video or animation of a robotic arm being controlled by a computer program with a graphical interface (buttons being clicked, sliders being moved).
- **Implementation:**
 - Show a brief clip of a robotic arm in a factory setting being controlled by an operator using a computer interface.
 - Alternatively, show a simple animation of a basic GUI with buttons and sliders controlling the movement of a virtual robotic arm.

- o Ask: "How do you think the person on the computer is telling the robot arm what to do? What kind of program do you think they are using?"

Strategies to use with this part:

- **Brainstorming:** Have students brainstorm different controls they might want on a computer program to control a robotic arm (e.g., buttons for specific movements, sliders for speed or angle).
 - **Real-World Connections:** Discuss examples of robots in factories or other applications that are likely controlled by computer programs with GUIs.
 - **Analogy:** Compare controlling the robot arm directly with Arduino code to using a remote control for a TV – the GUI is like the buttons on the remote, making it easier to send commands.
-

2. Explore:

Teacher Role:

- Introduce the activity as a way for students to visually plan the computer program they will use to control the robotic arm. Explain that sketching a "Graphical User Interface" (GUI) helps them think about what buttons, sliders, or other controls they will need.
- Guide students to access the Excalidraw software using the provided link or QR code. Offer a brief tutorial on the basic tools in Excalidraw (drawing shapes, adding text, lines) if needed.

For the teacher :

https://www.youtube.com/results?search_query=excalidraw+tutorial

- Encourage students to think about the different movements their robotic arm can perform (based on previous lessons) and what kind of on-screen controls would intuitively trigger those movements (e.g., a button labeled Gripper, Right & Lift, and a slider to control speed).
- Facilitate the brainstorming process by asking guiding questions like: "What are the most important actions the robot arm needs to do?", "How can we make the controls clear and easy to understand?", and "What kind of layout would make sense for these controls?"

- Provide support by circulating and offering suggestions as students sketch their interfaces. Encourage them to label their controls clearly and think about the flow of interaction.
- Connect this sketching activity to the "Watch it..." video about a simple scratch GUI, prompting students to think about the types of controls they saw and how they might adapt those ideas.

Expected From Students:

- Actively participate in the brainstorming process, thinking about the necessary controls for the robotic arm.
- Engage with the Excalidraw software to create a visual sketch of their proposed GUI.
- Experiment with different shapes (buttons, rectangles for layout), text labels, and arrangements of controls on their digital canvas.
- Clearly label the different controls in their sketch to indicate their intended function (e.g., "Grip Open," "Rotate Left," "Elbow Angle").
- Make connections between their sketched controls and the potential movements of the physical robotic arm.
- Consider ideas and control types presented in the "Watch it..." video while developing their own GUI sketch.

Preview with a Hook (Connecting to the Activity):

- **Strategy:** Start with a direct instruction that leads into the sketching.
- **Implementation:**
 - Say, "Imagine you have a blank computer screen in front of you. What buttons or sliders would you draw on that screen to make the robot arm move exactly how you want it to? Let's use Excalidraw to draw our ideas."

Strategies to use with this part:

- **Make Predictions (Visual):**
 - **Strategy:** Before using Excalidraw, have students quickly sketch a very basic control panel on paper.
 - **Implementation:**
 - Ask, "Quickly draw one or two shapes on your paper that you think could be buttons to control the robot arm. What would you label them?"

- Briefly share these simple paper sketches before moving to the digital tool.
 - **Set the Scene (Digital Tool Introduction):**
 - **Strategy:** Briefly demonstrate the basic tools of Excalidraw on a shared screen.
 - **Implementation:**
 - Show how to draw a rectangle (for a button), add text inside it, and draw a line.
 - Say, “Excalidraw is like a digital whiteboard where we can quickly sketch our ideas. Here are a few basic tools we can use.”
 - **Think-Pair-Share (Design Ideas):**
 - **Strategy:** Encourage students to discuss their initial GUI ideas with a partner before finalizing their Excalidraw sketch.
 - **Implementation:**
 - Ask, “Think about one specific movement of the robot arm (like making it wave). What kind of button or slider would you draw to control that? Share your idea with your partner.”
-

3. Assessment:

Teacher Role:

- Introduce the assessment as a way to check students' understanding of how programming concepts like "IF-THEN-ELSE" conditions and recording actions can make their robotic arm control smarter.
- Clearly explain that they need to choose the best answer for each question based on what they've learned so far in the lesson and their prior knowledge from the "Stepping Stone" section.
- Encourage them to think carefully about each option and why it might or might not be the correct answer.
- Facilitate a brief discussion after the assessment to review the correct answers and address any misunderstandings.

Expected From Students:

- Read and analyze each multiple-choice question.
- Apply their understanding of programming concepts and robotic arm control to select the correct answer.

- Demonstrate their knowledge of how code can introduce conditional behavior and automate sequences of movements.

Focus - Assessment Questions and Teacher Guidance:

- **Question 1: How can "IF-THEN-ELSE" conditions make your robotic arm smarter?**
 - A) By making the arm heavier.
 - B) By allowing the arm to follow specific instructions based on conditions.
 - C) By making the arm move faster.
 - D) By increasing the arm's battery life.
 - **Correct Answer: B) By allowing the arm to follow specific instructions based on conditions.**
 - **Teacher Guidance:** Explain that "IF-THEN-ELSE" is a fundamental programming concept that allows the robot to make decisions. For example, "IF a sensor detects an object, THEN grab it, ELSE move to the next position." This makes the robot's actions more intelligent and responsive to its environment. Options A, C, and D are not related to the logical control provided by conditional statements.
- **Question 2: How can you record and control your robotic arm's actions through code?**
 - A) By writing a program that tracks and executes specific movements.
 - B) By using a pencil to draw the movements.
 - C) By pressing random buttons.
 - D) By changing the arm's shape.
 - **Correct Answer: A) By writing a program that tracks and executes specific movements.**
 - **Teacher Guidance:** Connect this to the idea of creating sequences of instructions in code. Explain that they can write code to remember a series of movements (recording) and then tell the robot to perform those movements again (control/execute). Options B, C, and D are not methods of controlling a robot's actions through programming.

4. Practice:

Teacher Role:

- Introduce Python as a powerful but beginner-friendly programming language that is widely used in robotics and creating applications. Briefly explain some of its key features and why it's suitable for this task.
- Introduce Tkinter as a built-in library in Python that allows them to easily create graphical user interfaces (GUIs) with windows, buttons, sliders, and other interactive elements. Relate this back to the GUI designs they sketched in the "Explore" phase.
- Guide students through the very first steps of writing a Python script to create a basic Tkinter window. Provide a simple code example and explain each line:
 -
 - `import tkinter as tk`
 - `from tkinter import ttk # For themed widgets like sliders`
 -
 - `def send_command_to_arduino(command):`
 - `"""`
 - This function would normally send the 'command' to the Arduino.
 - For this simulation, we'll just print it.
 - `"""`
 - `print(f"Sending command to Arduino: {command}")`
 -
 - `def update_gripper(value):`
 - `gripper_value = int(float(value))`
 - `gripper_label.config(text=f"Gripper: {gripper_value}%")`
 - `send_command_to_arduino(f"gripper: {gripper_value}")`
 -
 - `def update_speed(value):`
 - `speed_value = int(float(value))`
 - `speed_label.config(text=f"Speed: {speed_value}%")`
 - `send_command_to_arduino(f"speed: {speed_value}")`
 -

- def update_left_right(value):
- lr_value = int(float(value))
- lr_label.config(text=f"Left/Right: {lr_value}")
- send_command_to_arduino(f"lr:{lr_value}")
-
- # Create the main window
- window = tk.Tk()
- window.title("Robotic Arm Control")
-
- # --- Gripper Control ---
- gripper_frame = ttk.LabelFrame(window, text="Gripper Control")
- gripper_frame.pack(padx=10, pady=10, fill="x")
-
- gripper_slider = ttk.Scale(
- gripper_frame,
- from_=0,
- to=100,
- orient="horizontal",
- command=update_gripper,
- length=200
-)
- gripper_slider.pack(padx=5, pady=5, fill="x")
- gripper_slider.set(0) # Initial value
-
- gripper_label = ttk.Label(gripper_frame, text="Gripper: 0%")
- gripper_label.pack(pady=2)
-
- # --- Speed Control ---
- speed_frame = ttk.LabelFrame(window, text="Speed Control")
- speed_frame.pack(padx=10, pady=10, fill="x")
-
- speed_slider = ttk.Scale(
- speed_frame,
- from_=0,
- to=100,

- orient="horizontal",
- command=update_speed,
- length=200
-)
- speed_slider.pack(padx=5, pady=5, fill="x")
- speed_slider.set(50) # Initial value
-
- speed_label = ttk.Label(speed_frame, text="Speed: 50%")
- speed_label.pack(pady=2)
-
- # --- Left & Right Control ---
- lr_frame = ttk.LabelFrame(window, text="Left & Right Control")
- lr_frame.pack(padx=10, pady=10, fill="x")
-
- lr_slider = ttk.Scale(
- lr_frame,
- from_=-90, # Assuming -90 to +90 degrees for left/right
- to=90,
- orient="horizontal",
- command=update_left_right,
- length=200
-)
- lr_slider.pack(padx=5, pady=5, fill="x")
- lr_slider.set(0) # Initial value (center)
-
- lr_label = ttk.Label(lr_frame, text="Left/Right: 0")
- lr_label.pack(pady=2)
-
- # Start the Tkinter event loop
- window.mainloop()

Expected From Students:

- Understand that Python is a programming language and Tkinter is used to create GUIs in Python.

- Write a basic Python script to create an empty Tkinter window.
- (If guided) Add a simple button widget to their Tkinter window.
- Run their Python script and observe the resulting GUI window.
- Begin to see the connection between writing Python code with Tkinter and creating the visual interface they designed.

Preview with a Hook (Connecting to Coding):

- **Strategy:** Start with an exciting first step into coding.
- **Implementation:**
 - Say, “Now we’re going to start making our own computer program to control the robot arm! The first step is to open a Python environment and type these few lines of code to create the main window of our control panel.” (Provide the basic Tkinter window code). “Run this code and see what happens – you’ve just created your first GUI!”

Strategies to use with this part:

- **Step-by-Step Guidance:** Break down the initial Python and Tkinter code into very small, manageable steps.
 - **Visual Demonstration:** Show students how to open a Python interpreter or IDE and run the code. Display the resulting window on a projector.
 - **Pair Programming:** Encourage students to work in pairs to type the code and troubleshoot any initial errors.
 - **Positive Reinforcement:** Celebrate the creation of even a basic window as a significant first step.
-

5. Showcase:

Teacher Role:

- Use this page to celebrate the students' progress and the successful application of their learning. Emphasize that they have taken a significant step towards controlling robots with computer programs.
- Connect back to the initial "Let's Think" question about developing a computer program to control the robotic arm and its benefits in factory operations. Discuss how Adam and Laila's success in the story reflects the potential of their own work.

- Encourage students to reflect on the skills they have learned in this lesson: understanding Python and Tkinter, designing a GUI, and starting to code the interface.
- Facilitate a brief discussion where students can share their experiences with the coding process, any challenges they faced, and their initial ideas for the controls in their GUI.
- Highlight the "Now I can..." statements on the previous page (Image 2, though labeled 24) as a summary of their accomplishments in this lesson.

Expected From Students:

- Feel a sense of accomplishment for starting to code their GUI for the robotic arm.
- Understand the connection between their learning and the successful outcome depicted in the story.
- Reflect on the new skills they have acquired (Python, Tkinter, GUI design).
- Share their experiences and initial ideas about their GUI controls.
- Recognize the progress they have made towards controlling the robotic arm with a computer program.

Preview with a Hook (Celebrating Success):

- **Strategy:** Begin by highlighting the achievement shown in the image.
- **Implementation:**
 - Say, "Look at Adam and Laila! They successfully created a computer program to control the robotic arm and even helped the factory workers. You have also taken the first steps in doing something similar by learning about Python and starting to code your own GUI!"

Strategies to use with this part:

- **Class Sharing:**
 - **Strategy:** If students have made progress on their GUI code, allow a few volunteers to briefly share their basic window or initial button designs with the class.
 - **Implementation:** Provide a platform for quick screen sharing (if using digital tools) or have students describe their designs.
- **Think-Pair-Share (Reflection):**
 - **Strategy:** Have students reflect on their learning.

- o **Implementation:** Ask, “What was one new thing you learned about Python or Tkinter in this lesson? What is one idea you have for a button or slider on your control panel?” Have them share their thoughts with a partner.

- **Connect to Future:**
 - o **Strategy:** Briefly discuss what the next steps in controlling the robotic arm with their GUI might involve (adding more controls, linking them to actions, communicating with the Arduino).
 - o **Implementation:** Say, “Now that we have our basic window, in the next lessons, we will learn how to add more interactive controls and make them actually tell the robot arm what to do!”

